

VSCode C++ 개발환경 완성 강의

코드 작성 · Git 관리 · CMake 빌드 · 단위 테스트 · 정적 분석



VSCode
& MinGW



Git
& GitHub



CMake
빌드



GoogleTest
TDD



정적 분석
clang-tidy

🕒 총 강의 시간: 3시간 | 수준: 입문 ~ 초급 | Windows 기준

전체 강의 구성 (3시간)

파트	시간	제목	핵심 내용
1부	25분	VSCode + MinGW-w64 C++ 개발환경 구축	설치, IntelliSense, 첫 빌드/디버그
2부	25분	Git + GitHub + VSCode Git UI	이력 관리, push/pull, GitLens
3부	20분	CMake 빌드 시스템	멀티파일 빌드, Debug/Release
—	5분	휴식	—
4부	25분	GoogleTest + TestMate 단위 테스트	테스트 환경 구축, TDD 사이클
5부	25분	GoogleTest 작성법 + TDD 실습	매크로, Fixture, 실습
6부	30분	정적 분석 (C/C++ Extension, clang-tidy, cppcheck)	경고 제거, lint, 자동화
7부	5분	마크다운 + 마무리 Q&A	README 작성, 전체 정리

사전 준비 — 설치 목록

VSCode

code.visualstudio.com

MinGW-w64

winlibs.com or sourceforge.net

Git

git-scm.com/downloads

CMake

cmake.org/download

GoogleTest

github.com/google/googletest

LLVM/clang

releases.llvm.org/download.html

cppcheck

cppcheck.sourceforge.io

⚠ MinGW-w64: x86_64-posix-seh 버전 선택 → 압축 해제 후 mingw64\bin 을 시스템 PATH에 추가

1

VSCode C++ 개발환경 구축

MinGW-w64 · C/C++ Extension Pack · 첫 빌드 · 디버깅

🕒 00:00 ~ 00:25 (25분)

VSCode 인터페이스 구성

Activity Bar

사이드바 보기 전환
Explorer · Git · Extensions · Testing

Side Bar

Activity Bar 선택에 따라
파일 탐색기 · 소스 컨트롤 등 표
시

Editor Group

파일 편집 영역
탭·분할 편집 지원

Panel (하단)

터미널 · 디버그 콘솔
출력 · 문제 탭

Status Bar

현재 파일 정보
CMake 빌드 버튼 등

💡 Ctrl+Shift+P → Command Palette 로 모든 명령을 검색·실행할 수 있습니다

MinGW-w64 설치 & 환경변수 등록

- ① winlibs.com 또는 sourceforge 에서 x86_64-posix-seh 버전 다운로드
- ② 압축 해제 후 mingw64 폴더를 원하는 경로로 이동 (예: C:\mingw64)
- ③ 시스템 환경변수 Path 에 C:\mingw64\bin 추가
- (제어판 → 시스템 속성 → 고급 → 환경 변수 → Path 편집)
- ④ 새 터미널(cmd)에서 설치 확인

```
# 버전 확인
gcc -v
g++ -v
gdb --version
```

✓ gcc version 13.x.x (MinGW-w64) 출력되면 성공

💡 특정 C++ 표준으로 컴파일하려면 `g++ main.cpp -std=c++17` 처럼 버전을 명시합니다

C/C++ Extension Pack 설치

C/C++

IntelliSense · 자동완성
디버그 UI 지원

CMake

CMakeLists.txt
IntelliSense

CMake Tools

빠른 빌드/실행/디버그
상태바 통합

C++ TestMate

GoogleTest UI 탐색기
트리뷰로 테스트 관리

GitLens

커밋 그래프·인라인 Blame
(별도 설치)

설치 방법: Ctrl+Shift+X → 'C++' 검색 → C/C++ Extension Pack [Install] → Reload Window

첫 번째 C++ 프로젝트 — 빌드 & 디버그

- ① 탐색기에서 language_test 폴더 생성
- ② VSCode: File → Open Folder → language_test
- ③ cpp/src/main.cpp 파일 생성 후 코드 작성
- ④ F5 → 'C/C++: g++.exe build and debug active file' 선택
- ⑤ TERMINAL 패널에서 Hello World!!! 출력 확인

```
#include <iostream>

int main() {
    std::cout << "Hello World!!!"
               << std::endl;
    return 0;
}
```

자동 생성 파일	설명
.vscode/tasks.json	g++ 컴파일 옵션 (args: -std=c++17, -g 등)
.vscode/c_cpp_properties.json	IntelliSense 설정 (C++ 표준, includePath 등)
.vscode/launch.json	디버거 실행 설정 (gdb / lldb)

2

CMake 빌드 시스템

멀티파일 빌드 · Quick Start · Debug/Release

🕒 00:50 ~ 01:10 (20분)

CMake가 필요한 이유 & 설치

단일 파일 → F5 빌드 OK

파일이 여러 개라면?

g++ main.cpp 만으로는 Test.cpp가 컴파일되지 않습니다.

→ CMake로 해결

- ① cmake.org/download/ 에서 설치파일 다운로드
- ② Install Options →
- 'Add CMake to the system PATH for all users' 선택
- ③ 설치 완료 후 cmd 에서 확인



```
cmake --version    # cmake version 3.x.x
```

CMake 구성요소	설명
CMakeLists.txt	빌드 스크립트 (프로젝트 이름, 소스파일, 링크 라이브러리 정의)
build/ 폴더	CMake가 생성하는 Makefile 및 중간 산출물 (gitignore 처리 필수)
CMake Tools 확장	VSCode 상태바에서 클릭 한 번으로 빌드/실행/디버그

CMakeLists.txt 작성 & 빌드



```
cmake_minimum_required(VERSION 3.0.0)
project(test_cmake VERSION 0.1.0)

# 실행파일 + 소스파일 목록
add_executable(test_cmake
    cpp/src/main.cpp
    cpp/src/Test.cpp
)

# (선택) C++17 표준 명시
set_target_properties(test_cmake PROPERTIES
    CXX_STANDARD 17
    CXX_STANDARD_REQUIRED ON
)
```

단축키	동작
F7	CMake: Build
Shift+F5	실행 (디버그 없이)
Ctrl+F5	CMake: Debug
Ctrl+Shift+P → Configure	Makefile 재생성
Ctrl+Shift+P → Clean	빌드 산출물 정리

빌드 타입	특징
Debug	최적화 없음 + 디버그 정보
Release	최적화 + 배포용
RelWithDebInfo	최적화 + 디버그 정보

⚙ Status Bar 하단 [Debug ▼] 버튼으로 빌드 타입을 전환할 수 있습니다



휴식 — 5분

01:10 ~ 01:15

3

GoogleTest + TestMate 단위 테스트 구축

환경 설치 · CMakeLists.txt · TDD 사이클 개념

🕒 01:15 ~ 01:40 (25분)

단위 테스트 & TDD 개념

TDD 사이클


RED

실패하는 테스트 먼저 작성


GREEN

테스트 통과할 최소 코드 구현


REFACTOR

동작 유지하며 코드 정리

도구	역할
GoogleTest	C++ 단위 테스트 프레임워크 (Google)
C++ TestMate 확장	VSCode에서 테스트 트리뷰 표시
CTest (CMake 내장)	Testing 패널 연동
enable_testing()	CMakeLists.txt에서 CTest 활성화

💡 코드 작성 시 기능 구현만 완료하면 반쪽짜리입니다. TDD로 작동하는 깔끔한 코드를 작성하세요!

GoogleTest 환경 구축 — 프로젝트 구조 & CMake

프로젝트 폴더 구조

```
language_test/  
├── cpp/src/  
│   ├── Plus.h / Plus.cpp  
│   └── Minus.h / Minus.cpp  
├── test/  
│   ├── TestPlus.cpp  
│   └── TestMinus.cpp  
├── googletest-1.13.0/ ← 소스 복사  
└── CMakeLists.txt
```

```
cmake_minimum_required(VERSION 3.0.0)  
project(test_cmake VERSION 0.1.0)  
set(This UnitTester) # 실행파일명  
set(SrcLib SrcLib)   # 소스 라이브러리  
enable_testing()     # CTest 활성화  
  
set(SrcFiles cpp/src/Plus.cpp  
           cpp/src/Minus.cpp)  
add_library(${SrcLib} STATIC ${SrcFiles})  
add_subdirectory(googletest-1.13.0)  
  
add_executable(${This}  
              test/TestPlus.cpp test/TestMinus.cpp)  
target_link_libraries(${This}  
                      gtest_main ${SrcLib})  
include(GoogleTest)  
gtest_discover_tests(${This})
```

설치: Ctrl+Shift+X → 'C++ TestMate' 검색 → Install → Reload Window

4

GoogleTest 작성법 + TDD 실습

매크로 함수 · Fixture · 실습 시나리오

🕒 01:40 ~ 02:05 (25분)

GoogleTest 주요 검증 매크로

EXPECT_ 계열 (실패해도 계속 진행)

매크로	설명
EXPECT_TRUE(cond)	cond == true 검증
EXPECT_EQ(a, b)	a == b 검증
EXPECT_NE(a, b)	a != b
EXPECT_LT / GT	a < b / a > b
EXPECT_STREQ(s1,s2)	문자열 동등
EXPECT_FLOAT_EQ(a,b)	부동소수점 근사

ASSERT_ 계열 (실패 즉시 중단)

- ASSERT_EQ / ASSERT_TRUE 등 — EXPECT_ 와 동일한 이름
- 실패 시 해당 TEST 함수 즉시 종료
- 선행 조건이 충족되지 않으면 이후 검증이 무의미한 경우 사용

테스트케이스 구조



```
#include "gtest/gtest.h"
#include "../cpp/src/Plus.h"

TEST(TestPlus, BasicAdd) {
    EXPECT_EQ(Plus::Run(1, 2), 3);
    EXPECT_EQ(Plus::Run(-1, -1), -2);
}
```

Test Fixture — 공통 데이터 재사용

```

// Fixture 클래스 정의
class MyFixture : public ::testing::Test {
protected:
    MyVector m_MyData;

    void SetUp() override {    // 각 테스트 실행 전
        m_MyData.Add(1);
        m_MyData.Add(2);
    }
    void TearDown() override { // 각 테스트 실행 후
        // MyVector 소멸자에서 자동 정리
    }
};

TEST_F(MyFixture, Test1) {    // TEST → TEST_F 로 변경
    EXPECT_EQ(m_MyData.GetAt(0), 1);
}
TEST_F(MyFixture, Test2) {    // 독립 실행: Test1과 무관
    EXPECT_EQ(m_MyData.GetCount(), 2);
}
}
```

Fixture 호출 흐름

1. TEST_F 호출

2. Fixture 객체 생성

3. SetUp() 실행

4. 테스트 본문 수행

5. TearDown() 실행

6. Fixture 소멸자

7. → 다음 테스트 반복

5

정적 분석 — C/C++ Extension · clang-tidy · cppcheck

빌드 없이 코드 품질을 높이는 자동화 도구

🕒 02:05 ~ 02:35 (30분)

정적 분석이란?

프로그램을 실제로 실행하지 않고 소스코드를 분석하여 버그, 보안 취약점, 코딩 규칙 위반을 찾아냅니다.



C/C++ Extension (IntelliSense 분석)

VSCode 내장
실시간 경고 표시
별도 설치 불필요

에디터 저장 시 실시간



clang-tidy

LLVM 기반
정교한 규칙 검사
자동 수정 가능

커밋 전 CI/CD에서



cppcheck

전용 분석기
런타임 오류 탐지
경량 설치

빌드 자동화 포함

◆ 세 도구를 함께 사용하면 레이어드 방어(layered defense) 효과로 코드 품질이 크게 향상됩니다

C/C++ Extension — 실시간 IntelliSense 분석

- C/C++ Extension Pack 설치만으로 기본 활성화
- 파일 저장 시 빨간/노란 물결선으로 문제를 즉시 표시
- Problems 패널 (Ctrl+Shift+M) 에서 전체 오류.경고 목록 확인

c_cpp_properties.json 설정 예시

```
{
  "configurations": [{
    "name": "Win32",
    "compilerPath": "C:/mingw64/bin/g++.exe",
    "cppStandard": "c++17",
    "intelliSenseMode": "windows-gcc-x64",
    "includePath": ["${workspaceFolder}/**"]
  }]
}
```

settings.json 설정	설명
"C_Cpp.errorSquiggles": "enabled"	오류 물결선 표시
"C_Cpp.clang_format_fallbackStyle": "Google"	저장 시 포맷 스타일
"editor.formatOnSave": true	저장 시 자동 포맷
"C_Cpp.codeAnalysis.enabled": true	확장 코드 분석 활성화

💡 Ctrl+Shift+M → Problems 탭에서 프로젝트 전체 오류/경고를 한눈에 확인할 수 있습니다

clang-tidy — 설치 & VSCode 연동

- ① LLVM 설치: releases.llvm.org/download.html
- (Windows: LLVM-x.x.x-win64.exe 선택)
- ② 설치 중 'Add LLVM to the system PATH' 선택
- ③ 설치 확인: `clang-tidy --version`

settings.json 연동 설정

```
{
  "C_Cpp.codeAnalysis.clangTidy.enabled": true,
  "C_Cpp.codeAnalysis.clangTidy.path":
    "C:/Program Files/LLVM/bin/clang-tidy.exe",
  "C_Cpp.codeAnalysis.runAutomatically": true
}
```

.clang-tidy 설정 파일

```
# .clang-tidy (프로젝트 루트에 배치)
Checks: >
  clang-diagnostic-*,
  clang-analyzer-*,
  cppcoreguidelines-*,
  modernize-*,
  readability-*
WarningsAsErrors: ''
FormatStyle: google
```

검사 그룹	내용
clang-diagnostic-*	컴파일러 경고와 동일한 수준
cppcoreguidelines-*	C++ Core Guidelines 준수
modernize-*	C++11/14/17 현대적 문법 제안

clang-tidy — 주요 경고 유형 및 자동 수정

modernize-use-nullptr

Before: `int* p = NULL;`

After: `int* p = nullptr;`

modernize-use-auto

Before: `std::vector<int>::iterator it = v.begin();`

After: `auto it = v.begin();`

cppcoreguidelines-init-variables

Before: `int x; // 초기화 없음`

After: `int x = 0;`

readability-identifier-naming

Before: `void myfunction() {}`

After: `void myFunction() {}`

modernize-use-override

Before: `virtual void f();`

After: `void f() override;`

⚡ clang-tidy --fix 옵션을 사용하면 지원되는 경고를 자동으로 수정할 수 있습니다

clang-tidy — 커맨드라인 사용법

① 기본 실행



```
# 단일 파일 분석
clang-tidy main.cpp -- -std=c++17

# 경고 자동 수정
clang-tidy --fix main.cpp -- -std=c++17
```

② compile_commands.json 활용



```
# CMakeLists.txt에 추가
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

# build 폴더 생성 후 분석
clang-tidy -p build/ cpp/src/main.cpp
```

검사 그룹	주요 내용	용도
clang-diagnostic-*	컴파일러 경고 수준 검사	기본 경고 포착
cppcoreguidelines-*	C++ Core Guidelines 준수 검사	코딩 규칙 적용
modernize-*	nullptr-auto-override 등 현대화 제안	C++11/14/17 전환
readability-*	네이밍·가독성 규칙 검사	코드 품질 향상



CMAKE_EXPORT_COMPILE_COMMANDS=ON 설정 시 헤더 경로와 컴파일 플래그를 자동 인식합니다

cppcheck — 설치 & VSCode 연동

- ① 설치: cppcheck.sourceforge.io
- ② 시스템 PATH 추가 후 확인: `cppcheck --version`
- ③ VSCode 확장: 'C/C++ Advanced Lint' 검색 후 설치
- ④ settings.json 에 cppcheck 경로 설정

settings.json 설정 (C/C++ Advanced Lint)

```
{
  "c-cpp-flylint.cppcheck.enable": true,
  "c-cpp-flylint.cppcheck.executable":
    "C:/Program Files/Cppcheck/cppcheck.exe",
  "c-cpp-flylint.cppcheck.language": "c++"
}
```

커맨드라인 직접 실행

```
# 기본 분석
cppcheck --enable=all --std=c++17 cpp/src/

# XML 리포트 생성
cppcheck --enable=all --xml
      --output-file=report.xml cpp/src/

# 특정 검사만 활성화
cppcheck --enable=warning,performance cpp/src/
```

--enable 옵션	설명
warning	잠재적 버그 경고
performance	성능 문제
portability	이식성 문제
style	코딩 스타일
all	모든 검사 활성화

cppcheck — 탐지 가능한 주요 오류 유형

메모리 오류

- ▶ `nullPointerDereference` – 널 포인터 역참조
- ▶ `memleak` – 메모리 누수 (`delete` 누락)
- ▶ `doubleFree` – 이중 해제
- ▶ `useAfterFree` – 해제 후 사용

버퍼/배열 오류

- ▶ `bufferAccessOutOfBounds` – 배열 범위 초과
- ▶ `arrayIndexOutOfBounds` – 인덱스 초과
- ▶ `writeOutsideBufferSize` – 버퍼 크기 초과 쓰기

초기화 & 타입

- ▶ `uninitvar` – 초기화되지 않은 변수 사용
- ▶ `uninitMemberVar` – 멤버 변수 미초기화
- ▶ `invalidPointerCast` – 잘못된 포인터 캐스트

성능 & 스타일

- ▶ `postfixOperator` – `i++` 대신 `++i` 권장
- ▶ `redundantAssignment` – 불필요한 대입
- ▶ `unusedFunction` – 미사용 함수

⚠ `cppcheck`는 실제 런타임 오류를 빌드 전에 찾아낼 수 있어 디버깅 시간을 크게 줄여줍니다

정적 분석 도구 비교 & 통합 전략

	C/C++ Extension	clang-tidy	cppcheck
분석 방식	IntelliSense 기반	LLVM AST 분석	전용 엔진
실행 시점	실시간 (타이핑 중)	저장/커밋 시	빌드 자동화
강점	즉각적 피드백	규칙 기반 현대화 제안	런타임 오류 탐지
자동 수정	일부	✅ --fix 지원	제한적
VSCode 통합	기본 내장	settings.json 설정	확장 별도 설치
설치 난이도	★ (자동)	★★ (LLVM 설치)	★★★ (설치 + 설정)

권장 통합 전략

코딩 중: C/C++ Extension 으로 실시간 경고 확인 → **커밋 전:** clang-tidy 규칙 검사 → **CI/CD:** cppcheck 자동화 분석

CMake에 정적 분석 자동화 통합

```

cmake_minimum_required(VERSION 3.0.0)
project(my_project)

# clang-tidy: 빌드 시 자동 실행
find_program(CLANG_TIDY clang-tidy)
if(CLANG_TIDY)
    set(CMAKE_CXX_CLANG_TIDY
        "${CLANG_TIDY}";
        "--config-file=${CMAKE_SOURCE_DIR}/.clang-tidy")
    message(STATUS "clang-tidy enabled")
endif()

# cppcheck: 커스텀 타겟으로 실행
find_program(CPPCHECK cppcheck)
if(CPPCHECK)
    add_custom_target(cppcheck
        COMMAND ${CPPCHECK} --enable=all
        --std=c++17 ${CMAKE_SOURCE_DIR}/cpp/src
        COMMENT "Running cppcheck...")
endif()

```

실행 방법

```

# clang-tidy: F7 빌드 시 자동 실행

# cppcheck: 별도 타겟으로 실행
cmake --build build --target cppcheck

```

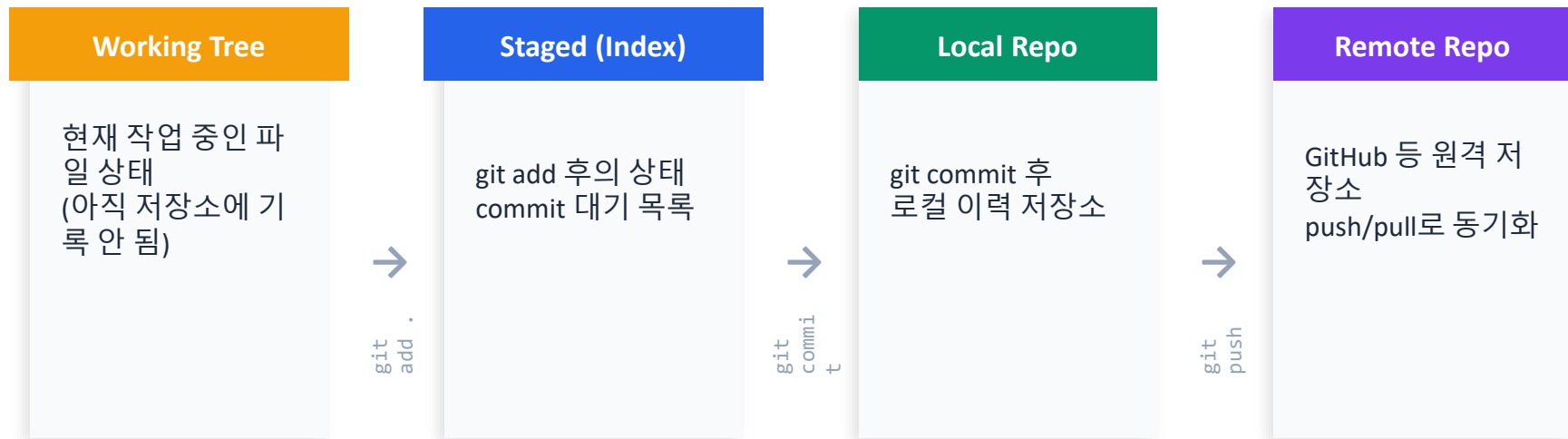
전체 정적 분석 워크플로우

- 1 코딩 중 — C/C++ Extension 실시간 경고
- 2 저장 시 — clang-tidy 경고 + 자동수정 제안
- 3 빌드 시 — CMAKE_CXX_CLANG_TIDY 자동 실행
- 4 CI/CD — cppcheck --enable=all 자동화

6

init · add · commit · push · pull · GitLens

🕒 00:25 ~ 00:50 (25분)



명령어	설명
<code>git init</code>	현재 폴더를 Git 관리 대상으로 초기화
<code>git add .</code> / <code>git status</code>	변경 파일 staged 전환 / 상태 확인
<code>git commit -m "메시지"</code>	staged 파일을 이력에 기록
<code>git log --oneline</code>	커밋 이력 요약 출력

VSCode Git UI + GitHub 연동

- ① Activity Bar → Source Control (Ctrl+Shift+G)
- ② Initialize Repository 버튼으로 Git 초기화
- ③ 변경 파일 옆 + 버튼 → Staged Changes 이동
- ④ 메시지 입력 후 Commit ✓ 버튼 클릭
- ⑤ ... 메뉴 → Undo Last Commit (최근 커밋 취소)



```
# GitHub 원격 연결
git remote add origin
https://github.com/user/repo.git

# 첫 pull (이력 병합)
git pull origin master --allow-unrelated-histories

# push (upstream 설정)
git push --set-upstream origin master
```

.gitignore 권장 내용



```
.vscode/
*.exe *.o
build/
```



GitLens 확장 설치 시 커밋 그래프·인라인 Blame·파일 히스토리를 시각적으로 확인할 수 있습니다

7

마크다운 + 마무리 Q&A

README 작성법 · 전체 워크플로우 정리

🕒 02:35 ~ 02:40 (5분)

마크다운 핵심 문법

문법	결과
# H1 / ## H2 / ### H3	헤더 (H1/H2/H3)
굵게 / *기울임*	굵게 / 기울임
[링크](URL)	하이퍼링크
![대체텍스트](이미지URL)	이미지 삽입
`인라인` / ```블록```	코드 강조
> 인용	인용문
- 항목 / 1. 항목	비순서 / 순서 목록
col1 col2 / --- ---	표 작성

.gitignore 예시

```

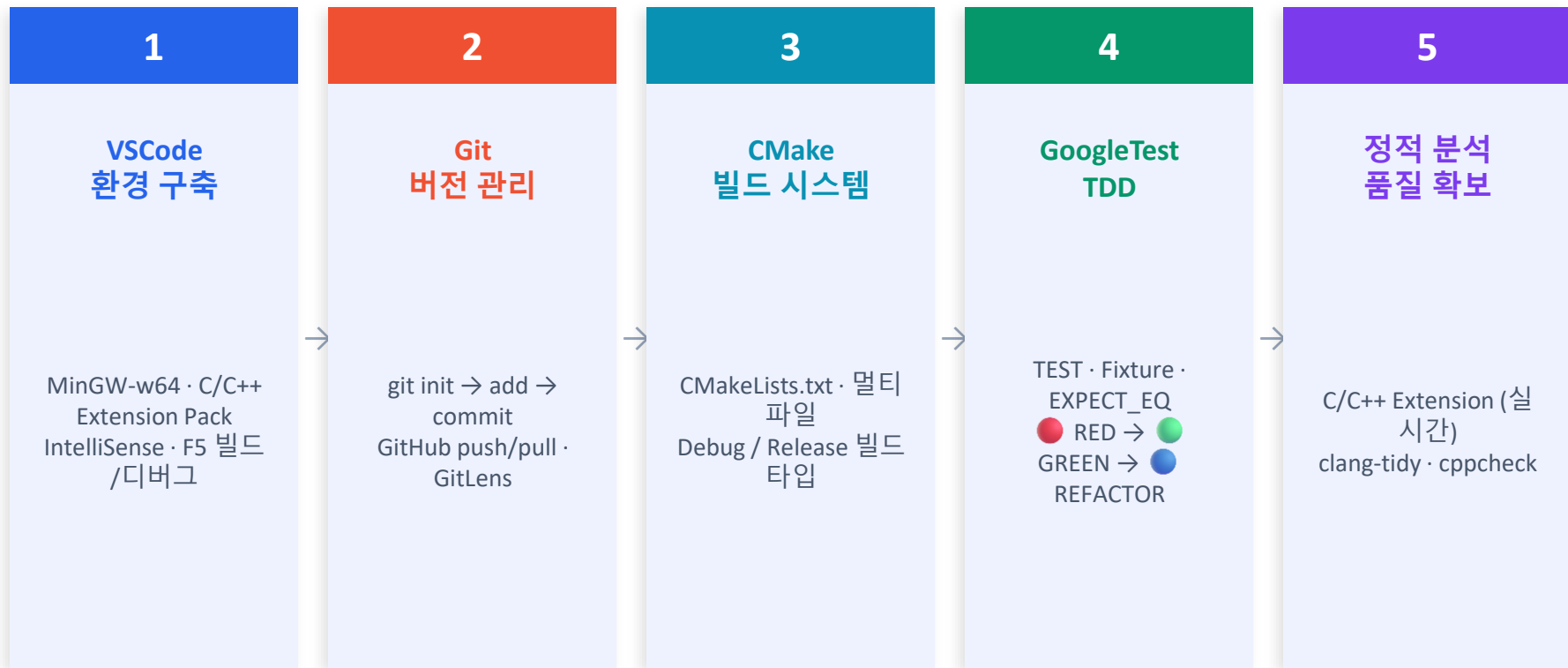
# 빌드 산출물
build/
*.exe *.o *.a
# VSCode 설정
.vscode/

```

README.md 권장 구성

- # 프로젝트명
- ## 개요 — 무엇을 하는 프로젝트인지
- ## 빌드 방법 — cmake, make 명령어
- ## 테스트 실행 — ctest 또는 ./UnitTester
- ## 정적 분석 — clang-tidy / cppcheck 실행법

전체 개발 워크플로우 정리



핵심 단축키 & 참고 자료

단축키	기능
F5	디버그 시작
F7	CMake 빌드
Shift+F5	실행 (디버그 없이)
Ctrl+F5	CMake 디버그
F9	중단점 설정/해제
F10 / F11	Step Over / Step Into
Ctrl+Shift+P	Command Palette
Ctrl+Shift+X	Extensions 열기
Ctrl+Shift+G	Source Control (Git)
Ctrl+Shift+M	Problems 탭 (정적 분석 결과)

참고 자료

강의 블로그

tango1202.github.io/categories/dev-setting/

GoogleTest 공식

google.github.io/googletest/

clang-tidy 체크 목록

clang.llvm.org/extra/clang-tidy/checks/

cppcheck 공식

cppcheck.sourceforge.io

CMake 공식

cmake.org/documentation/

VSCode C++ 공식

code.visualstudio.com/docs/languages/cpp

수고하셨습니다! 🎉

Clean Code That Works!!!

- ✓ VSCode + MinGW-w64 C++ 개발환경
 - ✓ Git + GitHub 소스 이력 관리
 - ✓ CMake 멀티파일 빌드 시스템
 - ✓ GoogleTest + TestMate TDD
- ✓ C/C++ Extension · clang-tidy · cppcheck 정적 분석

tango1202.github.io/categories/dev-setting/